



Formation Spring

1

Introduction

Tour de Table

➤ Présentez-vous :

- Qui êtes vous
- D'où venez vous
- Qu'attendez-vous de cette formation
- Quel est votre niveau / vécu en Java



Organisation

➤ Horaires

- Matin : 9h30 – 12h15
- Après Midi : 13h30 – 17h30

➤ Signalez vos impératifs

- Train, avion, ...



Sommaire

4

- Introduction
- Prise en main de Spring
- Le cycle de vie du conteneur Spring
- Bonnes pratiques de configuration de beans
- Injection de dépendance avec les annotations
- Programmation orientée aspect (AOP)
- Accès aux données et JDBC avec Spring
- Gestion des transactions avec Spring
- JUnit et Spring
- Intégration de Spring avec JPA et Hibernate
- Spring Data
- Spring MVC – JSP / Servlet
- Spring MVC – Web Services Rest
- Présentation de Spring Security
- Tests JUnit avec Mock et sans Mock pour les web services
- Spring WebSocket
- Spring JMX

Introduction

5

- ✓ Présentation
- ✓ Historique
- ✓ Objectifs
- ✓ Notion de conteneur léger
- ✓ Ecosystème Spring
- ✓ IoC : Inversion de contrôle
- ✓ Inversion de dépendance
- ✓ Apports du Spring



Présentation

- Spring est un projet open-source fondé en 2003
 - Dirigé par la société SpringSource

- Spring est composé de nombreux modules indépendants
 - Injection de dépendances
 - Programmation par aspects
 - Gestion déclarative des transactions
 - Utilitaires pour les frameworks les plus utilisés de la sphère Java
 - ❑ Hibernate, Struts, JSF...
 - ...

Historique

- Mars 2004 : Spring 1.0
 - Juillet 2004 : sortie du livre "J2EE without EJB"
 - Septembre 2004 : Spring 1.1
 - Mai 2005 : Spring 1.2
 - Octobre 2006 : Spring 2.0
 - Novembre 2007 : Spring 2.5
 - Décembre 2009 : Spring 3.0
 - 2011 Spring 3.2 : Dernière version de maintenance en 31/12/2016
 - Java 5 minimum
 - 2014 : Spring 4.1
 - Java 6 minimum (support complet Java 8)
 - JEE 7 : JMS 2.0, JTA 1.2, JPA 2.1, Servlet 3.+
 - 2015 : Spring 4.2
 - Java 7 minimum
 - Gestion des évènements
- ← VMWare achète SpringSource (2009/08)
- ← Création de Pivotal (2013/04)

Spring Boot 1

Historique

➤ 2016 : Spring 4.3 (Java 1.6 à 1.8) : Dernière version de maintenance en 31/12/2020

Spring Boot 2

➤ 2017-07 : Spring 5.0

- WebFlux : faire très rapidement des contrôleurs un peu comme les routes en angular
- Support Java 9
- JEE 7 - 8

➤ 2018-09 : Spring 5.1

- Java 8 Minimum
- Supporte le Java 11
- Support officiel de JUnit 5

Spring Boot 2.2

➤ 2019-09 : Spring 5.2

- Java 8 Minimum
- Supporte le Java 13
- Support de Kotlin 1.3

➤ Réintégration de Pivotal
dans VMWare (2019/08)

Historique

Spring Boot 2.3

- 2020-05 : Spring 5.3 (dernière release Spring Boot 2.7 avec une fin de support pour 06/2029)
 - Support de Kotlin 1.4 et du Java 14
 - Intégration de R2DBC (version réactive de Spring JDBC)
 - Support plus complet de WebFlux/reactive

Spring Boot 3

- 2022-11 : Spring 6.0
 - Java 17 minimum
 - Jakarta EE 9 (pour le Web et le JPA)



*Broadcom achète
VMWare (05/2022)*

Spring Boot 3.2

- 2023-12 : Spring 6.1 (dernière release Spring Boot 3.5 avec une fin de support pour 06/2032)
 - Support de Kotlin 1.9 et Java 21
 - JdbcClient
 - Amélioration de la gestion des Threads
 - Complément sur les Observables (télémétrie)
 - Jakarta EE 10

Historique

Spring Boot 4

- 2025-11 : Spring 7 : Fin du support 31/12/2027
 - Encourage l'usage du Java 21 (mais toujours compatible Java 17)
 - Jakarta EE 11
 - Redécoupage des Spring Boot Starter = Grosses modifications dans les outils de builds
 - Ajout des Services Clients (@HttpExchange) = Consommateurs simplifiés d'API Rest
 - Meilleure granularité sur la gestion des métriques (Open Telemetry)
 - Changement de version Jackson = Changement sur les Import dans le code
 - Changement de version JUnit = passe à JUnit 6 (très proche du 5)

- 2026 – 06 : Spring Boot 4.1.y
 - Ajout du gRPC (partie serveur et client + tests)
 - Ajout du SSRF (Server Side Request Forgery)
 - Gestion native des log rotatif de Log4J
 - Amélioration de la gestion de Kafka
 - Amélioration du Spring Data (prise en compte de Resdis)

Historique

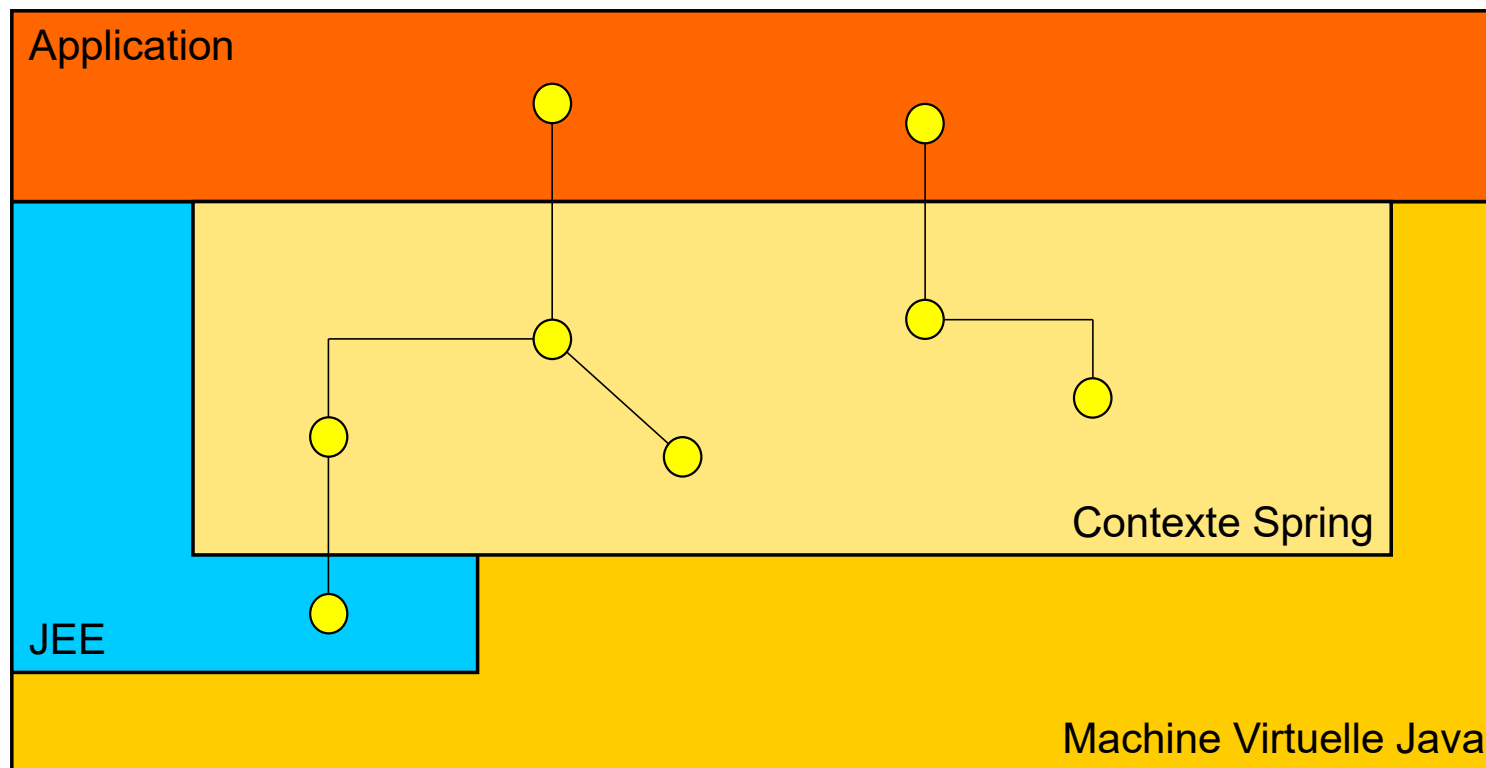
- Toujours consulter le site de l'éditeur afin d'évaluer les versions à venir ainsi que la fin du support
 - <https://github.com/spring-projects/spring-framework/wiki/Spring-Framework-Versions>
- Ne pas confondre
 - **Spring** (tout seul) ou Spring Core : le noyau du Spring
 - **Spring Boot** : un module **COMPLEMENTAIRE** du Spring Core. Il n'est **PAS** indispensable, mais il est **TRES** pratique

Objectifs du framework Spring

- Toute application d'entreprise met en œuvre un ensemble d'objets collaborant entre eux pour répondre à des requêtes métier
- Les associations entre objets peuvent se faire :
 - par construction
 - par un système de "lookup" (JNDI, Service Locator)
- Avec Spring les composants ne sont plus responsables d'établir leurs associations avec les autres composants de l'application
 - La création de ces associations est prise en charge par un **conteneur**

Assemblage avec Spring

13



Notion de "conteneurs légers"

- Nés du rejet de la complexité des conteneurs JEE, principalement ceux des EJBs
- Initiatives largement supportées par l'Open Source
- PicoContainer et NanoContainer
 - <http://www.picocontainer.org/>
- Spring
 - <http://www.springframework.org/>
- Excalibur (ex Avalon)
 - <http://excalibur.apache.org/>
- Hivemind
 - <http://jakarta.apache.org/hivemind/>
- Google Guice
 - <http://code.google.com/p/google-guice/>

Notion de "conteneurs légers"

➤ Le concurrent direct actuel de Spring est Quarkus

➤ <https://quarkus.io/>



➤ Permet de faire du déploiement natif

➤ Spring ne sait officiellement le faire qu'à partir de sa version 6 (fin 2022)

➤ Respecte les normes / annotations JEE

➤ ex: @Get à la place de @GetMapping

Comparatif Spring vs Quarkus

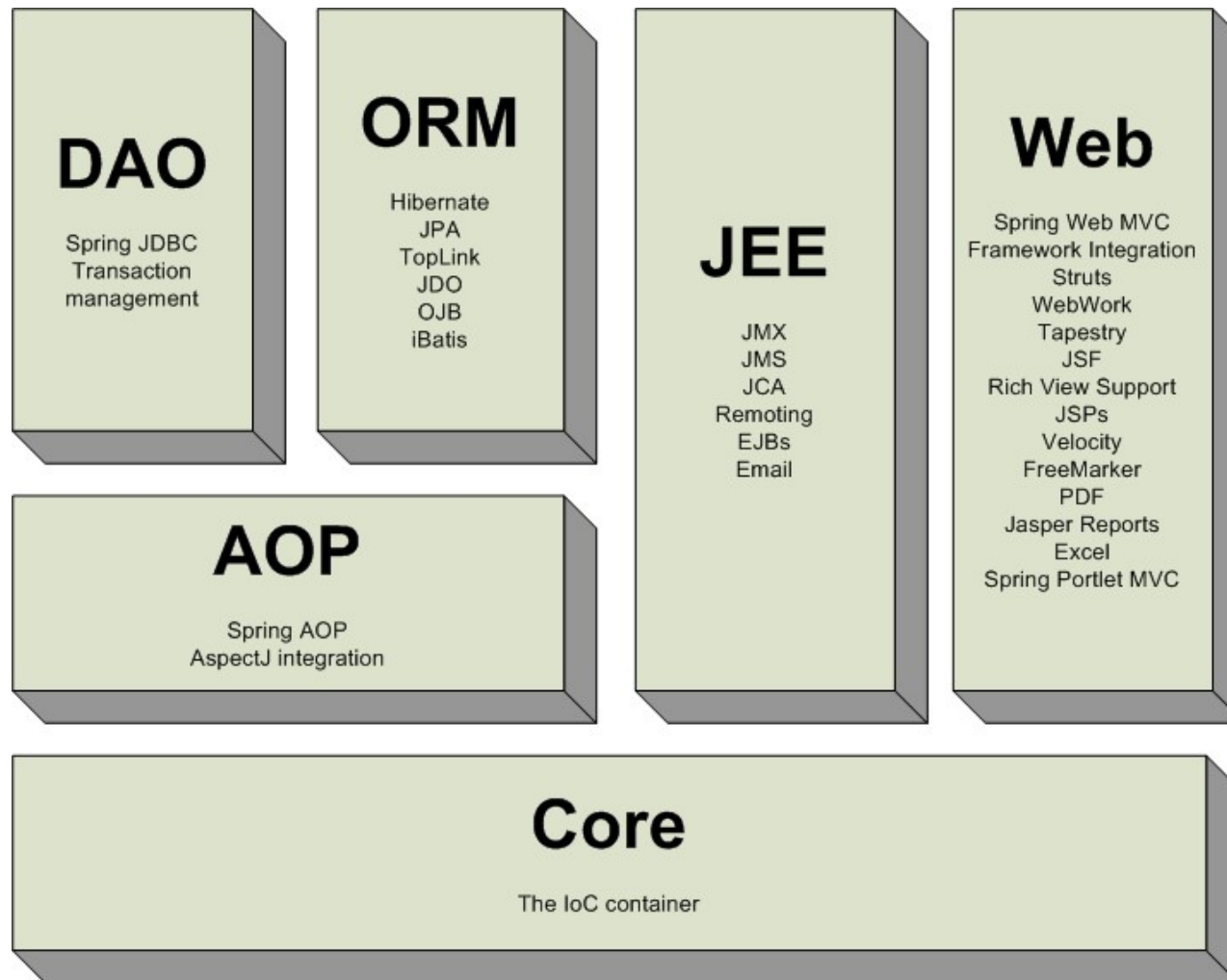
- <https://www.baeldung.com/spring-boot-vs-quarkus>
- Article de mai 2023
- **Finally, all of the versions have robust performance when compared, at least for our example. The difference is so tiny that we can say they have similar performance.** Of course, we can argue that the JVM versions handled the heavy load better in terms of throughput, while consuming more resources, and the native versions consumed less. However, this difference may not even be relevant depending on the use case.

Spring et les composants logiciels

- Spring, comme les autres conteneurs légers du marché, simplifie l'intégration d'autres frameworks
 - Présentation (Struts, Tapestry, JSF, etc..)
 - Persistance (Hibernate, JDO, OJB, etc..)
 - Sécurité (Spring Security)

- Spring n'impose **aucune contrainte** sur les frameworks que vous pouvez utiliser
 - Sauf potentiellement leur version minimale

Les différents modules Spring



Les projets du portfolio Spring (1/2)

➤ Tous disponibles sur : <https://spring.io/projects>

➤ **Spring Tool Suite** (anciennement STS)

➤ <https://spring.io/tools>

➤ Version customisée d'Eclipse



➤ **Spring Security** (anciennement Acegi Security)

➤ Gestion de la sécurité dans une application

➤ Approche déclarative et transverse



➤ **Spring For GraphQL**

➤ <https://spring.io/projects/spring-graphql>

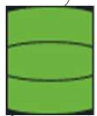
➤ Permet de faire des WebServices Rest en GraphQL

Les projets du portfolio Spring (2/2)



➤ **Spring Batch**

- Gestion de batch (traitements de masse)
- Au sens production (job, step, reprise sur erreurs, ...)



➤ **Spring Data (Spring 4+)**

- Mécanisme automatisée pour la gestion des repository (DAO)
- Peut aller jusqu'à l'automatisation des Web Service REST

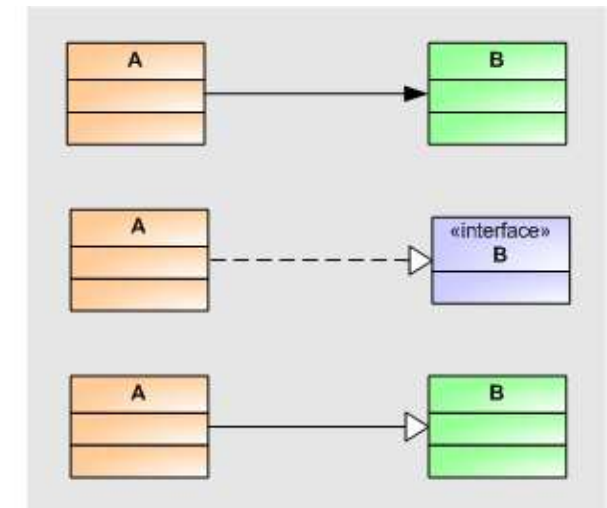
Les différents types de liens

- Plus largement, un composant **A** dépend d'un composant **B** si nous ne pouvons pas compiler le composant A sans le composant B.

A utilise B

A implémente l'interface B

A hérite de B



- La notation UML exprime la dépendance de A envers B par une flèche en pointillés :



Conséquences sur les dépendances

- Les dépendances ne permettent pas :
 - d'installer A sans installer B
 - ❑ donc de réutiliser A sans réutiliser B
 - une modification de B (changement de signature de méthode, ...)
 - ❑ peut entraîner une modification de A
 - ❑ une recompilation de A
- Il y a perte de réutilisation et de modularité
- Une mauvaise gestion des dépendances entraîne pour l'application...
 - Rigidité
 - ❑ Application difficile à modifier car chaque changement affecte d'autres parties du logiciel
 - Fragilité
 - ❑ Une modification entraîne des effets de bords : BUG
 - Immobilité
 - ❑ Difficulté de reprendre une partie de l'application car elle ne peut pas être indépendante des autres parties

Injection de dépendances (1/2)

➤ Dépendance sur la classe concrète

```
public class Order {  
    public void write() {  
        OrderStateXMLWriter w = new OrderStateXMLWriter();  
        w.write(this.getState());  
    }  
}
```

- Utilisation de l'opérateur « **new** ».
- La classe « Order » est directement dépendante de l'implémentation du « Writer ».
- Couplage fort entre classes
 - ❑ Refactoring systématique si l'on désire modifier l'implémentation ou le mécanisme d'instanciation.
 - ❑ Impossible de choisir l'implémentation en cours d'exécution de l'application.
- Emploi de solutions alternatives
 - ❑ Utilisation de classes de type « Factory ».
 - ❑ Emploi du mécanisme de « lookup » (JNDI ou non).

Injection de dépendances (2/2)

➤ Dépendance uniquement par l'interface

```
public class Order {  
  
    private IWriter writer;  
  
    public IWriter getWriter() {  
        return writer;  
    }  
    public void setWriter(IWriter writer) { this.writer = writer; }  
  
    public void write() { this.writer.write(this.getState()); }  
}
```

➤ La classe « Order » travaille uniquement sur l'interface

- ❑ Elle n'a pas connaissance de l'implémentation de « IWriter ».
- ❑ Elle ne doit pas connaître le mécanisme d'instanciation de « IWriter ».

➤ Comment fournir l'implémentation ?

- Nous avons besoin d'une entité externe pour gérer les dépendances.
- Plus précisément, d'un outil réalisant l'injection de dépendances.

IoC : Inversion of Control

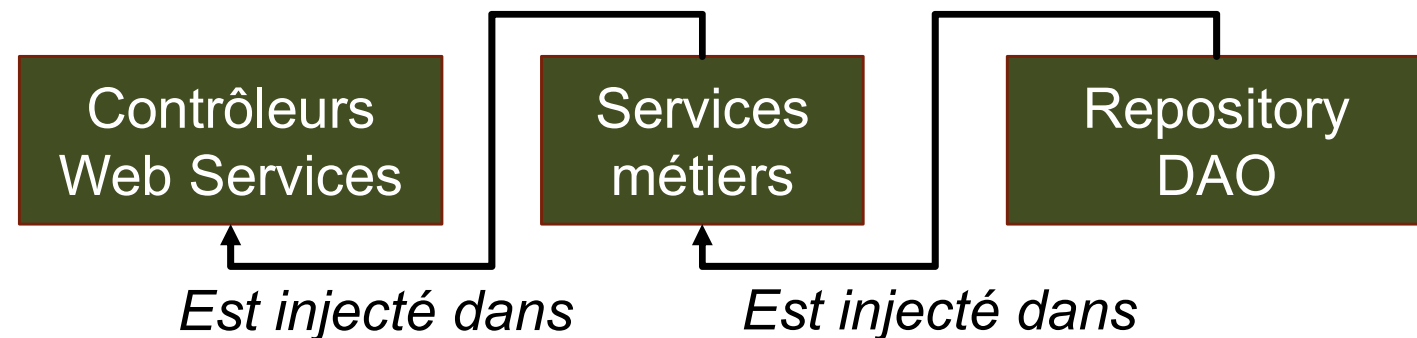


- **Définition** : c'est un patron d'architecture commun à tous les frameworks (ou cadre de développement et d'exécution). Il fonctionne selon le principe que le flot d'exécution d'un logiciel n'est plus sous le contrôle direct de l'application elle-même mais du framework ou de la couche logicielle sous-jacente.
- L'inversion de contrôle est un terme générique. Selon le problème, il existe différentes formes, ou représentation d'IoC, le plus connu étant l'injection de dépendances (dependency injection) qui est un patron de conception permettant, en programmation orientée objet, de découpler les dépendances entre objets.

Présentation Spring : Ioc

➤ Dans notre cas :

- On fabrique les différents éléments (Contrôleurs / Services / Repository)
- C'est Spring qui gère leurs relations via les informations que l'on va lui donner
 - ❑ C'est lui qui fera de l'injection de dépendance



Rappels sur les couches :

- Un **Contrôleur** : Objet bête mais technologiquement typé (souvent WEB), qui reçoit une demande (ex: requête HTTP) puis passe la main aux services. Retourne un résultat sous la forme d'un flux de données ou d'une page web. Exemple : une servlet, un Web Service.
- Un **Service** : Objet intelligent technologiquement non typé qui va porter toutes les fonctionnalités du projet. Représente la partie fonctionnelle du projet. Souvent issue des Uses Cases.
- Un **Repository** (ou DAO) : Objet qui porte la responsabilité d'interagir avec la base de données. C'est le seul qui peut envoyer des requêtes à la base. Il est responsable du CRUD.

Rappels sur les couches

➤ Un **Contrôleur** :

- On y injecte zéro ou plusieurs services.

➤ Un **Service** :

- On y injecte zéro ou plusieurs services.
- On y injecte zéro ou plusieurs repository.

➤ Un **Repository** (ou DAO) :

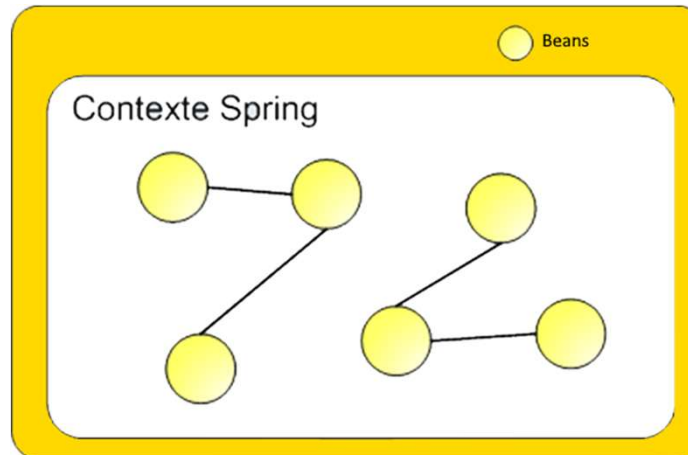
- On y injecte des objets utilitaires qui simplifient l'accès à la base (gestion des connexions, des requêtes, ...).

Les apports de Spring (1/3)

- Spring est un outil permettant de satisfaire les dépendances entre objets par injection
 - Configuration centralisée – programmation déclarative des dépendances.
 - Spring est non intrusif vis à vis du code.
 - Une application est un assemblage de composants.
- Un **bean** = Un objet **géré** par Spring
- Spring gère automatiquement le cycle de vie de tous les beans
 - Instanciation automatique des beans quel que soit l'ordre de déclaration.
 - Destruction des beans selon la configuration.
- Spring respecte le principe Hollywood – Ne nous appelez pas, nous vous appellerons (« Don't call us, we'll call you ! »)

Les apports de Spring (2/3)

- Spring est un **conteneur de beans**. Ces derniers évoluent au sein d'un contexte spécifique – « `ApplicationContext` »
 - C'est au sein de ce contexte que les beans existent.
 - Si le contexte disparaît, les beans disparaissent également.



Les apports de Spring (3/3)

- Spring offre un ensemble d'outils pour l'aide au développement
 - Couche d'abstraction pour les accès en base de données par JDBC.
 - Intégration de briques logicielles externes : Hibernate par exemple.
 - Gestion déclarative des transactions.
 - Support pour l'AOP.
 - Framework Spring MVC pour le développement d'applications Web.
- Les outils reposent tous sur le principe de l'injection de dépendances
- Spring offre une programmation orientée « composant »
- Spring respecte le principe de l'architecture nTiers

Spring et Maven (sans Spring Boot)

➤ Dépendances Maven simplifiées :

```
96 <dependencyManagement>
97   <dependencies>
98     <!-- Afin d'éviter d'indiquer tous les éléments du Spring -->
99     <dependency>
100       <groupId>org.springframework</groupId>
101       <artifactId>spring-framework-bom</artifactId>
102       <version>${version.spring}</version>
103       <scope>import</scope>
104       <type>pom</type>
105     </dependency>
```

Le <dependencyManagement>
ne vous épargnera pas de
placer des dépendances dans
<dependencies>

- Tous les éléments du **core** spring seront cohérents par rapport à la version indiquée
 - Les modules complémentaires seront à ajouter en plus (spring-ws, spring-data, spring-security, ...)

Spring et Maven (sans Spring Boot)

- En fonction de ce que vous voulez faire, vous ajouterez dans `<dependencies>` (*groupId* et *artifactId*) mais sans préciser les versions (qui sont gérées par le `<dependencyManagement>`)
- Exemple : Spring-context

```
</dependencyManagement>  
  
<dependencies>  
  <dependency>  
    <groupId>org.springframework</groupId>  
    <artifactId>spring-context</artifactId>  
  </dependency>  
</dependencies>
```

Configuration Spring

- Vous avez **trois** possibilités pour configurer votre projet Spring
 - Par **fichiers XML** (Spring version ≤ 3)
 - Par **classes de configuration** (Spring version ≥ 3)
 - Par **fichiers properties/Yaml** = Spring Boot (Spring version ≥ 4)
- Il est possible de mixer ces trois modes de configuration (mais ce n'est pas recommandé)
 - Cependant, on mixe souvent deux modes :
 - ❑ Spring boot + @Configuration